An Iterative and Reproducible Heart-Disease Classification Workflow Using the UCI Cleveland Dataset

Shyamkumar T
School of Computer Science and Engineering
VIT-AP University, Amaravati, India
CodeAlpha Machine Learning Internship – Task 4

***Abstract*—This paper documents the end-to-end development of an educational binary classifier for the CodeAlpha Disease Prediction task. The workflow was built iteratively in Google Colab using the UCI Heart Disease dataset (303 records and 13 features). The original five-level target, `num`, was mapped to a binary target by assigning value 0 to class 0 and values 1–4 to class 1, following the dataset's documented experimental convention. The pipeline includes stratified hold-out splitting, heterogeneous preprocessing, five-fold stratified cross-validation, comparison of Logistic Regression, radial-basis-function Support Vector Machine (SVM), Random Forest, and XGBoost, followed by grid-search tuning. Logistic Regression with $C = 0.1$ and balanced class weights was selected by mean cross-validation ROC-AUC. On the 61-record held-out split, it achieved 88.52% accuracy, 83.87% precision, 92.86% recall, 88.14% F1-score, 84.85% specificity, and 0.9654 ROC-AUC. The model was serialized with its preprocessing pipeline and verified after reload; a standalone CSV prediction script subsequently passed smoke tests, including a test with reversed feature order. The report preserves the chronological modeling and packaging decisions, reports observed outputs, and distinguishes benchmark performance from clinical validity. Because the dataset is small and historical and no external validation was performed, the implementation is an educational demonstration rather than a diagnostic tool.**

***Index Terms*—Heart disease classification, tabular machine learning, Logistic Regression, SVM, Random Forest, XGBoost, cross-validation, model persistence, reproducibility.**

I. Introduction

Medical prediction tasks often use structured records containing demographic measurements, exercise-test results, and coded diagnostic variables. The assigned project called for a disease-prediction system built from tabular data, making classical supervised classifiers more appropriate to the specified data form than the CNN or sequence-model architectures covered in the user's Deep Learning coursework. The work described here uses the Cleveland subset accessed through the UCI Machine Learning Repository, whose dataset description reports 303 instances, 13 features, and a target taking values from 0 to 4; prior experiments commonly collapse the target into absence (0) versus presence (1–4) [1], [2].

The practical aim was broader than fitting a classifier. The work proceeded through data inspection, target construction, preprocessing, baseline evaluation, model comparison, hyperparameter search, held-out evaluation, exploratory interpretation, serialization, command-line prediction, and repository packaging. The process was conducted iteratively: each notebook output informed the next step, and later packaging issues were addressed in follow-up conversation. The paper therefore records both model results and the engineering decisions that made the model reusable.

The contribution is a transparent record of this specific implementation and its outputs, not a new clinical algorithm. Scikit-learn provided the model and preprocessing API [3]; SVM, Random Forest, and XGBoost were included as comparison families grounded in their established literature [4]–[6]. Model selection and reporting caveats are discussed in light of known selection bias in cross-validation-based model search and clinical prediction-model reporting guidance [7], [8].

II. Dataset and Problem Formulation

A. Data source and feature schema

The dataset was retrieved in Colab via `ucimlrepo.fetch_ucirepo(id=45)`. The downloaded matrix had shape 303×13; the target matrix contained one column, `num`. The 13 features were `age`, `sex`, `cp`, `trestbps`, `chol`, `fbs`, `restecg`, `thalach`, `exang`, `oldpeak`, `slope`, `ca`, and `thal`. The UCI metadata describes these as a mixture of categorical and integer/continuous features and records missing values in `ca` and `thal` [1].

B. Target transformation

The original target counts observed in the notebook were 164 records with `num`=0, 55 with value 1, 36 with value 2, 35 with value 3, and 13 with value 4. The implementation created

$$y_i = \mathbb{1}(\mathrm{num}_i > 0), \tag{1}$$

where $y_i = 0$ is labelled "disease absent according to dataset target" and $y_i = 1$ is labelled "disease present according to dataset target." This produced 164 class-0 records (54.13%) and 139 class-1 records (45.87%). The wording intentionally refers to the historical dataset target; the labels should not be read as clinical diagnoses for new patients.

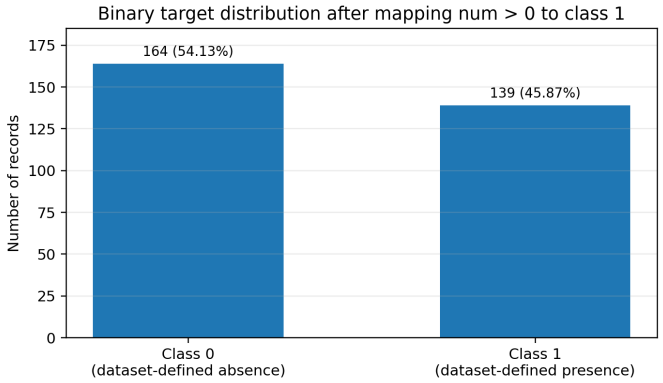


Fig. 1. Observed class distribution after converting the original target to a binary outcome.

TABLE I
NUMERICAL FEATURE SUMMARY FROM THE 303 RECORDS

Feature	Mean	Std. dev.	Min	Max
age	54.44	9.04	29	77
trestbps	131.69	17.60	94	200
chol	246.69	51.78	126	564
thalach	149.61	22.88	71	202
oldpeak	1.04	1.16	0.0	6.2

C. Exploratory data checks

No target values were missing. Six feature cells were missing: four in `ca` and two in `thal`. There were no missing values in the remaining 11 feature columns. The descriptive statistics and discrete-category frequency tables were inspected before selecting preprocessing. Table I summarizes the five measurement-style features and Table II shows counts of the coded variables, including the six missing values.

III. METHODOLOGY

A. Feature grouping and preprocessing

The code grouped `age`, `trestbps`, `chol`, `thalach`, and `oldpeak` as numerical features. Median imputation and `StandardScaler` were applied to this group. The remaining columns (`sex`, `cp`, `fbs`, `restecg`, `exang`, `slope`, `ca`, and `thal`) were treated as categorical/discrete and passed through most-frequent imputation and one-hot encoding. This was a deliberate modeling choice for this experiment; alternative encodings, especially for the vessel count `ca`, were not investigated.

The numerical and categorical branches were combined using `ColumnTransformer` and placed within a scikit-learn `Pipeline`. Placing the imputation, scaling, encoding, and estimator inside the pipeline ensures that transformation parameters are learned within each training fold instead of being fitted on validation or test observations, following scikit-learn guidance on avoiding data leakage [9]. Unknown encoded categories were configured to be ignored at inference time.

TABLE II
OBSERVED CATEGORY COUNTS FOR CODED/DISCRETE FEATURES

Feature	Counts by value
sex	0: 97; 1: 206
cp	1: 23; 2: 50; 3: 86; 4: 144
fbs	0: 258; 1: 45
restecg	0: 151; 1: 4; 2: 148
exang	0: 204; 1: 99
slope	1: 142; 2: 140; 3: 21
ca	0: 176; 1: 65; 2: 38; 3: 20; missing: 4
thal	3: 166; 6: 18; 7: 117; missing: 2

B. Train/test split and cross-validation

A stratified 80/20 split with `random_state=42` created 242 training records and 61 held-out test records. The training set contained 131 class-0 and 111 class-1 records; the test set contained 33 class-0 and 28 class-1 records. A five-fold `StratifiedKFold` with shuffling and random state 42 was used for initial model comparison and grid search. The scoring metrics were accuracy, precision, recall, F1-score, and ROC-AUC.

C. Models and hyperparameter search

The baseline models were Logistic Regression, RBF-kernel SVM, Random Forest, and XGBoost. Initial configurations were: Logistic Regression with `max_iter=2000`; RBF SVM with $C = 1$; Random Forest with 300 trees and fixed random seed 42; and XGBoost with 150 estimators, depth 2, learning rate 0.03, subsampling 0.9, column sampling 0.9, and $\lambda = 1.0$. These are initial comparison settings, not claimed optimal defaults.

Grid search then explored Logistic Regression $C \in \{0.01, 0.1, 1, 10, 100\}$ and class weighting in {none, balanced}; SVM $C \in \{0.1, 1, 10\}$, $\gamma \in \{\text{scale}, 0.1\}$ and the same class-weight options; Random Forest depth in {unlimited, 4, 8} and minimum leaf size in {1,2,4}; and XGBoost estimator count in {100,200}, depth in {2,3}, and learning rate in {0.03,0.1}. GridSearchCV refitted each model using mean cross-validation ROC-AUC. The selected Logistic Regression configuration was $C = 0.1$ with balanced class weights; its best mean cross-validation ROC-AUC was 0.9028. The rendered Colab output truncated the selected parameter dictionaries for SVM, Random Forest, and XGBoost, so this report records their actual search spaces and scores but does not invent their exact winning configurations.

IV. EXPERIMENTAL RESULTS

A. Logistic Regression baseline

The initial Logistic Regression baseline achieved mean five-fold accuracy of 0.847, precision of 0.877, recall of 0.783, F1-score of 0.825, and ROC-AUC of 0.902. Accuracy varied modestly across folds (standard deviation 0.011); recall averaged lower than precision. These baseline results served as a point of comparison, not as the final model.

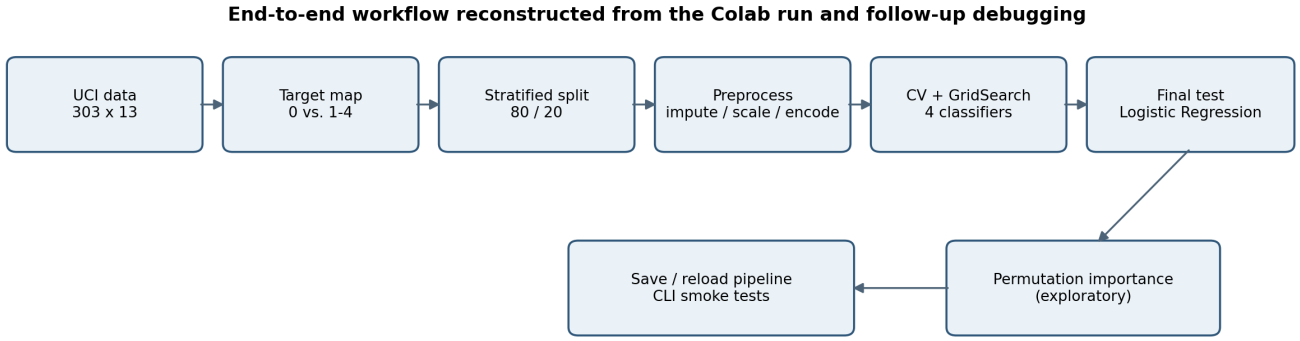


Fig. 2. Workflow reconstructed from the Colab notebook and the subsequent packaging/debugging conversation.

TABLE III
INITIAL FOUR-MODEL COMPARISON: MEAN AND STANDARD DEVIATION ACROSS FIVE TRAINING FOLDS

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	0.847 ± 0.011	0.877 ± 0.061	0.783 ± 0.051	0.825 ± 0.008	0.902 ± 0.016
SVM (RBF)	0.814 ± 0.026	0.830 ± 0.083	0.765 ± 0.089	0.789 ± 0.030	0.888 ± 0.027
Random Forest	0.814 ± 0.026	0.797 ± 0.030	0.801 ± 0.090	0.796 ± 0.041	0.894 ± 0.037
XGBoost	0.814 ± 0.027	0.813 ± 0.056	0.783 ± 0.100	0.792 ± 0.038	0.887 ± 0.029

B. Model comparison before tuning

Logistic Regression led the initial comparison in accuracy, precision, F1-score and ROC-AUC. Random Forest had the highest mean recall (0.801), though its fold-to-fold recall standard deviation was 0.090. These results motivated testing all four estimators under a consistent grid-search procedure rather than choosing a model from one metric alone. The concerns around repeatedly optimizing a finite-sample cross-validation criterion are documented in the model-selection literature [7].

C. Tuned model results

After search, Logistic Regression retained the highest mean ROC-AUC (0.903) and F1-score (0.813, narrowly ahead of XGBoost at 0.811). XGBoost led in mean accuracy (0.834) and precision (0.855), while tuned Logistic Regression led in mean recall (0.810). The metrics are close and their fold variability is not negligible, so selection was based on the stated ROC-AUC criterion, not on a claim that Logistic Regression dominates in every dimension.

D. Held-out test evaluation

The selected Logistic Regression pipeline was refitted by GridSearchCV on the complete training split and then evaluated on the 61-record test split. Table V shows the reported metrics. The confusion matrix was $\begin{bmatrix} 28 & 5 \\ 2 & 26 \end{bmatrix}$ when rows denote actual labels [0,1] and columns denote predicted labels [0,1]. Thus 54/61 records were classified correctly, with five false positives and two false negatives.

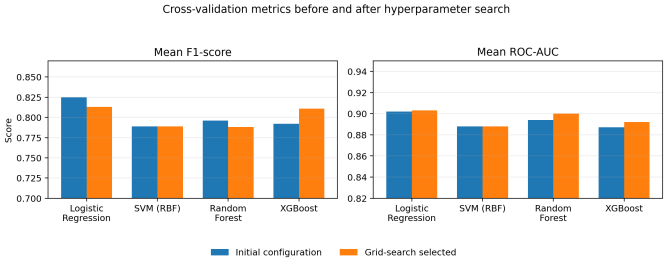


Fig. 3. Mean F1-score and ROC-AUC before and after grid search. Values are reconstructed from the notebook's cross-validation tables.

The positive-class recall of 0.9286 indicates that 26 of 28 positive-labelled test records were detected at the default 0.5 threshold. Specificity of 0.8485 indicates 28 of 33 class-0 records were correctly classified. The test ROC-AUC exceeded the cross-validation estimate (0.9654 versus 0.9028); on such a small test sample, this difference can arise from split composition and should not be taken as evidence of generalization to another population. The test set was not used to tune the decision threshold.

E. Exploratory permutation importance

Permutation importance was computed 50 times per feature using test-set ROC-AUC as the scoring function. This method estimates how much a fitted model's score decreases when feature values are shuffled; it quantifies reliance of this fitted model on the observed sample rather than causal influence [10]. The largest mean decreases were for `cp` (0.0283), `ca` (0.0261), and `oldpeak` (0.0243). Some importance standard deviations were substantial relative to the means, consistent

TABLE IV
GRID-SEARCH-SELECTED CONFIGURATIONS: CROSS-VALIDATION MEAN AND STANDARD DEVIATION

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC
Logistic Regression	0.830 ± 0.017	0.823 ± 0.043	0.810 ± 0.068	0.813 ± 0.023	0.903 ± 0.017
SVM (RBF)	0.814 ± 0.024	0.830 ± 0.074	0.765 ± 0.080	0.789 ± 0.027	0.888 ± 0.024
Random Forest	0.814 ± 0.041	0.824 ± 0.063	0.764 ± 0.089	0.788 ± 0.048	0.900 ± 0.030
XGBoost	0.834 ± 0.036	0.855 ± 0.069	0.783 ± 0.089	0.811 ± 0.042	0.892 ± 0.025

TABLE V
HELD-OUT TEST RESULTS FOR SELECTED LOGISTIC REGRESSION

Metric	Score
Accuracy	0.8852
Precision	0.8387
Recall / sensitivity	0.9286
Specificity	0.8485
F1-score	0.8814
Balanced accuracy	0.8885
ROC-AUC	0.9654
Average precision	0.9622

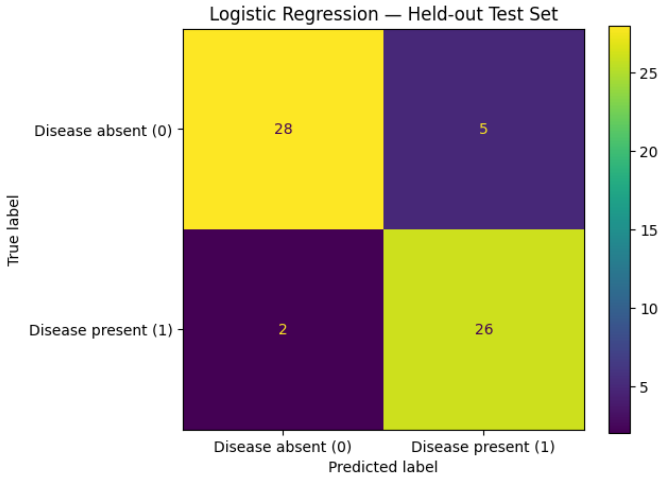


Fig. 4. Confusion matrix from the held-out test run as displayed in the Colab conversation.

with uncertainty from a 61-record evaluation set. Negative mean importances for `restecg`, `chol`, `trestbps`, and `age` do not establish that these variables lack medical relevance. No features were removed and no model was changed based on this post-hoc analysis.

V. CHRONOLOGICAL DEVELOPMENT AND DEBUGGING RECORD

The implementation was not a single uninterrupted training script; it was refined in response to observed outputs and packaging failures. The numbered log below reconstructs the verified sequence from the Colab export and follow-up conversation. A displayed output is reported as observed evidence, while an action that was only recommended is not claimed as completed.

Stage 1: Dataset acquisition and audit. The first Colab cell fetched UCI dataset ID 45 and returned 303 records, 13 feature columns, and the target column `num`. Six feature values were missing (four in `ca`, two in `thal`); the target had no missing values. Raw feature and target copies were retained.

TABLE VI
TOP PERMUTATION-IMPORTANCE VALUES

Feature	Mean decrease	Std. dev.
cp	0.0283	0.0110
ca	0.0261	0.0165
oldpeak	0.0243	0.0083
thal	0.0124	0.0121
exang	0.0094	0.0076
sex	0.0040	0.0080
thalach	0.0034	0.0099
slope	0.0017	0.0053
fbs	0.0007	0.0008
restecg	-0.0005	0.0039
chol	-0.0008	0.0034
trestbps	-0.0021	0.0049
age	-0.0027	0.0023

Stage 2: Binary target and exploratory analysis. The transform `num > 0` produced 164 class-0 rows (54.13%) and 139 class-1 rows (45.87%). Numerical summaries and category counts were inspected before preprocessing.

Stage 3: Initial Logistic Regression baseline. Five-fold training cross-validation returned mean accuracy 0.847, precision 0.877, recall 0.783, F1-score 0.825, and ROC-AUC 0.902. These metrics established a baseline; they did not justify declaring a final model.

Stage 4: Initial four-model comparison. Logistic Regression led in initial accuracy, F1, and ROC-AUC, while Random Forest had the largest mean recall (0.801) but a recall standard deviation of 0.090. All four model families were compared under the same folds and scoring metrics.

Stage 5: Grid-search tuning. GridSearchCV selected Logistic Regression with $C = 0.1$ and balanced class weights; its best mean cross-validation ROC-AUC was 0.9028. Tuned XGBoost yielded the highest mean accuracy (0.834) and precision (0.855), while Logistic Regression retained the best mean ROC-AUC (0.903) and F1-score (0.813). The selected dictionaries for the other models were truncated in the displayed notebook output, so their search spaces and scores are reported without reconstructing unknown winning parameters.

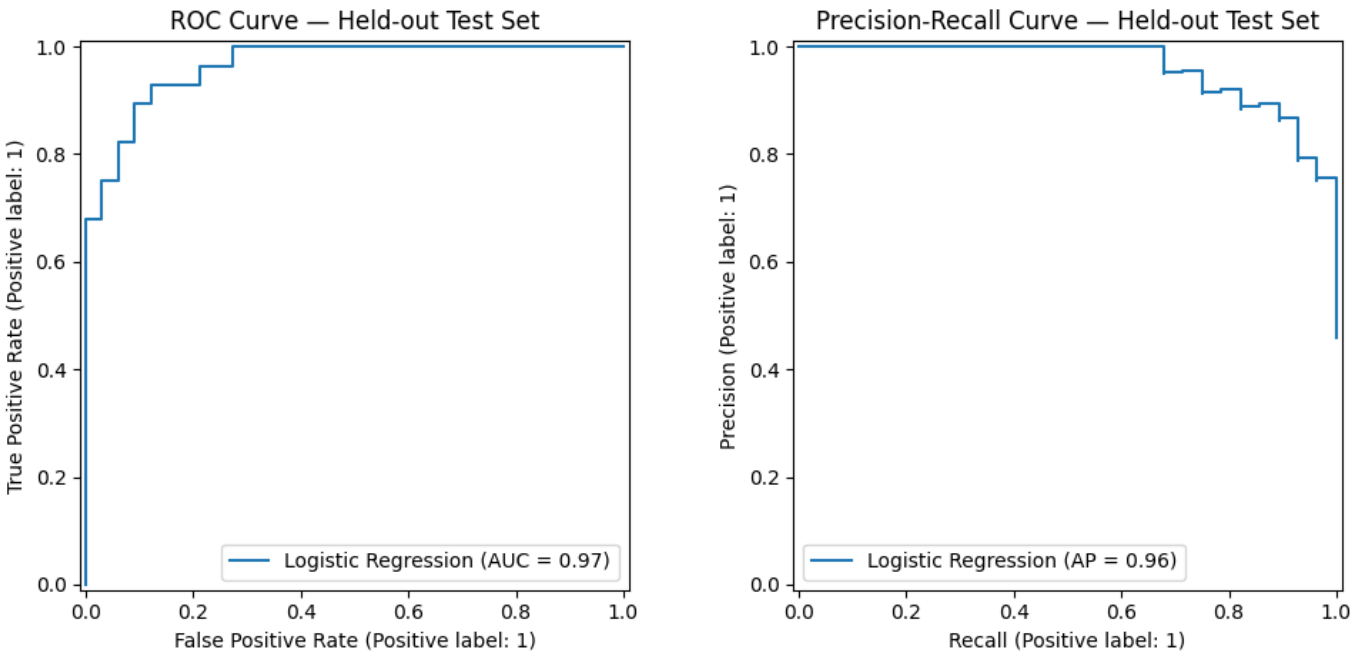


Fig. 5. Curves from the actual held-out run: ROC-AUC was 0.9654, displayed as 0.97 in the ROC legend; average precision was 0.9622, displayed as 0.96 in the precision–recall legend.

Stage 6: Held-out evaluation. On 61 test records, the selected model achieved accuracy 0.8852, precision 0.8387, recall 0.9286, specificity 0.8485, F1 0.8814, ROC-AUC 0.9654, and average precision 0.9622. Its confusion matrix was $[[28, 5], [2, 26]]$. The default 0.5 decision threshold was retained.

Stage 7: Post-hoc interpretation. Fifty permutation repeats per feature gave the largest mean ROC-AUC decreases for `cp` (0.0283), `ca` (0.0261), and `oldpeak` (0.0243). The results were treated as exploratory; no feature was removed and no retraining or threshold change followed.

Stage 8: Serialization. The entire preprocessing-plus-classifier pipeline was saved with `joblib` and reloaded. Predictions and probabilities matched before and after serialization. One training-row smoke-test returned label 1 and class-1 score 0.661.

Stage 9: CLI portability correction. The initial generated predictor expected its model in the project root, while the desired repository design placed the model in `models/`. The pasted version also appeared with escaped identifier/comment markup. A corrected complete script was installed with script-relative paths, the final model location, output under `outputs/`, feature-column alignment, and input-validation/error-handling checks.

Stage 10: End-to-end smoke test. The standalone script completed in a subprocess with return code 0 for five records. Input columns were deliberately reversed; predicted labels $[1, 0, 0, 0, 1]$ and scores $[0.6610, 0.1625, 0.3363, 0.3132, 0.6906]$ matched the fitted pipeline. A subsequent one-row sample predicted class 0 with score 0.459836 and wrote an output CSV.

Stage 11: Local Git setup. Windows PowerShell showed identical SHA-256 hashes for the root-level and `models/` model copies, so the duplicate root file was removed. The subsequent `git status` and `git add .` commands failed with "fatal: not a git repository." Initializing Git with `git init -b main`, committing, configuring the remote, and pushing were advised, but successful publication is not evidenced in the supplied conversation.

A. Serialization and command-line portability

The original package-writing cell in the Colab export initially wrote the model at the project root and generated a simple `predict.py`. During later review, the displayed source prompted concern because its path assumptions no longer matched the intended folder structure and the chat-rendered code contained escaped identifier markup. A full replacement script was installed. Its final path logic resolves the project directory from `Path(__file__).resolve().parent`, loads `models/heart_disease_logistic_pipeline.joblib`, reads `metadata.json`, aligns input columns to the stored schema, validates required fields, and writes to `outputs/predictions.csv` by default. It also preserves extra input columns while excluding them from inference,

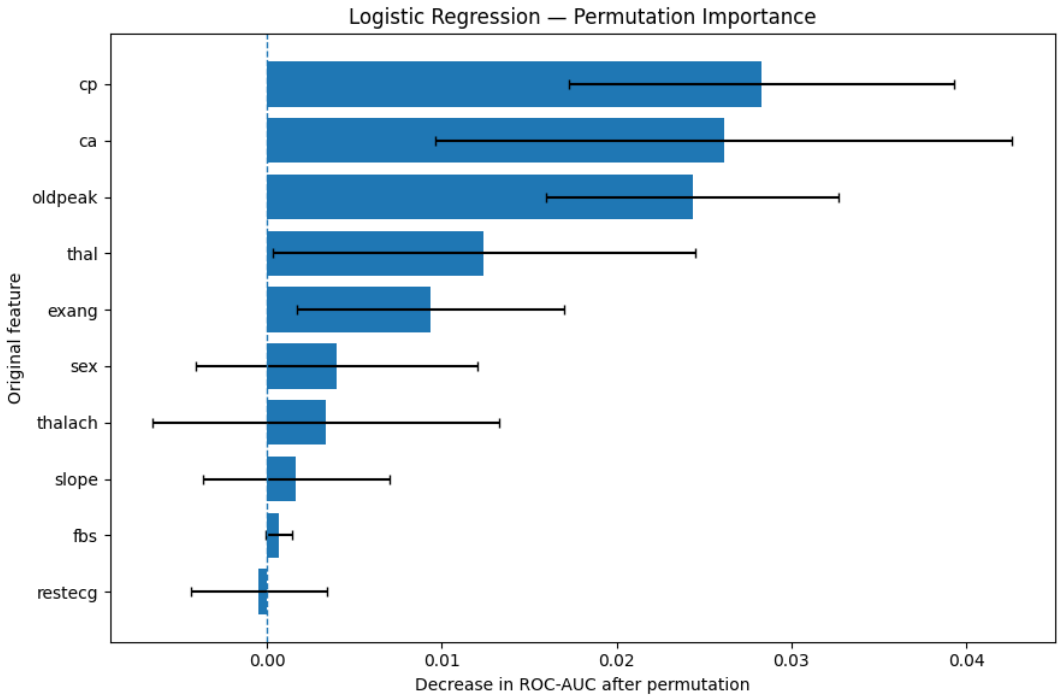


Fig. 6. Permutation-importance output from the notebook conversation, with mean ROC-AUC decrease and standard-deviation bars.

rejects non-numeric feature text, warns on missing feature values, and labels the score as an educational model output.

The successful five-row smoke test ran in a subprocess and compared the standalone script output against the fitted pipeline with exact label equality and probability agreement to an absolute tolerance of 10^{-12}. The final one-row test also confirmed that the revised script wrote an output CSV under the desired project structure. These are software correctness checks within the same runtime/environment, not independent validation of predictive performance on a new cohort.

B. Repository status at report preparation

The supplied Windows Explorer screenshot showed the project artifacts (examples, models, outputs, metadata, report CSVs, README, model card, references, requirements, predictor script, and Colab PDF). The notebook export was recommended but was not visible in that screenshot. A later PowerShell transcript confirmed the duplicate model hashes matched and the root-level copy was removed. It also showed that the local folder had not yet been initialized as a Git repository. The report therefore describes GitHub publication as pending verification; it does not claim that the repository push succeeded.

VI. Discussion

The experiment demonstrates a complete classical-ML workflow using a small tabular dataset and reproducible pipeline components. Logistic Regression was a defensible selection under the chosen cross-validation ROC-AUC criterion. Its test recall was relatively high (0.9286), but this estimate is based on only 28 positive-labelled test records; two false negatives were observed. Five false positives among 33 class-0 records also limit the interpretation of a positive label. The 0.9654 test ROC-AUC and 0.9622 average precision are encouraging on this split but should not be treated as stable estimates of real-world or clinical performance.

The large gap between cross-validation and test ROC-AUC could reflect the random composition of this small hold-out set. The same test split was subsequently used to compute permutation importance; although that analysis did not cause any model, threshold, or feature-selection change, it is a post-hoc analysis on the evaluation sample and should be disclosed as such. Future rigorous work would use repeated/nested resampling for development estimates and independent external data for final validation. Cawley and Talbot explain how model-selection optimization can overfit the finite-sample selection criterion itself [7]. TRIPOD+AI emphasizes transparent reporting of model development, evaluation, data sources, and intended use [8].

The scope is a learning artifact, not a clinical system. The dataset is historical, small, and originates from a particular clinical context. It cannot justify generalized claims about current patients or populations. Model probabilities were not calibrated for clinical risk, decision thresholds were not selected using decision analysis, and no assessment of clinical utility, subgroup calibration, or external validity was performed. The project documentation correctly warns against diagnostic or treatment use.

VII. Conclusion

This project progressed from raw UCI data inspection to a fitted, evaluated and packaged heart-disease classification pipeline. The dataset contained 303 observations and 13 features; target binarization produced 164 class-0 and 139 class-1 records. Four classifier families were compared through stratified cross-validation and grid search. Logistic Regression ($C = 0.1$, balanced class weights) was selected by mean five-fold ROC-AUC of 0.9028. On the held-out 61-row split, the model obtained 88.52% accuracy and ROC-AUC 0.9654, with 26 true positives, 28 true negatives, five false positives and two false negatives. The full pipeline passed model reload checks and a standalone CLI smoke test after paths and folder layout were corrected. The chronological record also reveals a remaining repository-management step: the local folder's Git initialization failed, and a successful GitHub push was not verified in the evidence available for this report. The project is technically reproducible at the tested CLI stage but remains an educational benchmark, not a clinically validated diagnostic model.

Acknowledgment

This report was reconstructed from the Google Colab PDF export dated 19 October 2026, outputs shared during the project conversation, and the final inspected `predict.py`. Dataset, software, algorithm and reporting-method references are cited below. No result absent from those materials is presented as an observed output.

References

[1] A. Janosi, W. Steinbrunn, M. Pfisterer, and R. Detrano, "Heart Disease," UCI Machine Learning Repository, dataset 45, donated June 30, 1988, DOI: 10.24432/C52P4X. [Online]. Available: https://archive.ics.uci.edu/dataset/45/heart+disease. Accessed: Oct. 19, 2026.

[2] R. Detrano *et al.*, "International application of a new probability algorithm for the diagnosis of coronary artery disease," *The American Journal of Cardiology*, vol. 64, no. 5, pp. 304–310, Aug. 1989, doi: 10.1016/0002-9149(89)90524-9.

[3] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[4] C. Cortes and V. Vapnik, "Support-vector networks," *Machine Learning*, vol. 20, pp. 273–297, 1995, doi: 10.1007/BF00994018.

[5] L. Breiman, "Random forests," *Machine Learning*, vol. 45, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.

[6] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.

[7] G. C. Cawley and N. L. C. Talbot, "On over-fitting in model selection and subsequent selection bias in performance evaluation," *Journal of Machine Learning Research*, vol. 11, pp. 2079–2107, 2010.

[8] G. S. Collins *et al.*, "TRIPOD+AI statement: Updated guidance for reporting clinical prediction models that use regression or machine learning methods," *BMJ*, vol. 385, p. e078378, Apr. 2024, doi: 10.1136/bmj-2023-078378.

[9] Scikit-learn developers, "Common pitfalls and recommended practices," scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/common_pitfalls/. Accessed: Oct. 19, 2026.

[10] Scikit-learn developers, "Permutation feature importance," scikit-learn documentation. [Online]. Available: https://scikit-learn.org/stable/modules/permutation_importance.html. Accessed: Oct. 19, 2026.